

ELI Orchestration and Agents — Full Topology

Contents

Two agent stacks (this is the key thing to understand)	1
Planning artifacts — 4 of them, only 1 drives execution	3
Where it is actually weak (corrected)	3
Recommended fixes (prioritized)	4
Update — 2026-06-09	5

Supersedes the earlier `agent_bus.md`, which only documented the parallel specialist bus and missed the real `AgentOrchestrator`. Read-only reference; nothing here changes behaviour.

Source files: - `eli/cognition/orchestrator.py` — the real orchestrator (12-stage pipeline) - `eli/cognition/agent_bus.py` — the parallel 14-specialist bus - `eli/kernel/engine.py` — wiring / dispatch gate - `eli/execution/execution_planner.py` — declarative plan model (UNUSED) - `eli/planning/task_planner.py` — planner shim (stub)

Two agent stacks (this is the key thing to understand)

ELI does **not** have one agent system. It has two, selected by reasoning mode and action type:

1. `AgentOrchestrator` — the real orchestrator (`orchestrator.py`)

The 12-stage cognitive pipeline. The engine calls it via `_run_internal_orchestrator` (`engine.py`:7265, 8982). Components:

- **`PlannerAgent.plan_retrieval()`** (`orchestrator.py`:53) — produces a **mode-aware retrieval plan**:
 - fast: keyword only, no FAISS/RAG, KG only if identity, 1 ReAct iter, skip HyDE
 - balanced (default): keyword + semantic + KG, RAG if doc query, 3 ReAct iters
 - deep: everything, large budgets, full HyDE, 3 ReAct iters
- **`OrchestratorMemoryAgent`** (`orchestrator.py`:131) — performs the retrieval: HyDE expansion → keyword/FTS5 + FAISS semantic + document RAG + KG → `hybrid_merge` → rerank. **Sequential** by design — the `llama_cpp` embedder is not thread-safe and segfaults under concurrent daemon-thread calls.
- **`ExecutorAgent`** (`orchestrator.py`:119) — thin wrapper over `executor_enhanced.execute`.

Flow inside `AgentOrchestrator.run()`: - **Non-CHAT actions** (`orchestrator.py`:539–740): dispatches the **AgentBus** for specialist evidence (line 561), then runs a **ReAct observation loop** (598–644): runs the executor, asks the loaded LLM ANSWER or TOOL:<action> <args>, and **chains to the next tool**, accumulating observations. 1 iter in fast mode, up to 3 otherwise. For “grounded synthesis” actions the observations are assembled into context and passed to the LLM; for direct actions the executor result is returned as-is. - **CHAT** (`orchestrator.py`:742–897): Stage 3 HyDE → Stage 4 planner → Stage 5/6/7 retrieval → Stage 8 merge → Stage 9 rerank → Stage 10 context assembly → Stage 10.5 persona handoff → Stage 11 generation. Private reasoning modes (`chain_of_thought`, `self_consistency`, `tree_of_thoughts`, `constitutional_ai`) hand off to `engine._run_chat_reasoning_loop` so the mode-specific algorithm runs. **Note: the AgentBus is NOT called on the CHAT path** — the orchestrator uses its own `OrchestratorMemoryAgent` retrieval instead.

2. AgentBus — the parallel 14-specialist fan-out (agent_bus.py)

15 agents in _ALL_AGENTS (agent_bus.py:3071), each a _BaseAgent subclass with name + timeout_s:

#	name	timeout_s	accesses
1	memory	5.0	SQLite + FTS5 + FAISS; self- gates (_eli_memory_sho
2	system	8.0	direct action execu- tion (SYSTEM_ACTIONS)
3	habit	3.0	user_patterns / habit tables
4	self_improvement	3.0	LoRA / self- tuning intro- spection
5	proactive	3.0	suggestion / antici- pation
6	frontier	5.0	frontier / aware- ness model
7	plugin	6.0	plugin registry (PLUGIN_ACTIONS)
8	capability	6.0	capability manifest
9	voice	5.0	TTS / voice subsys- tem
10	orchestrator	3.0	bus-level planner — emits a plan dict only
11	file_code	4.0	source tree / code intro- spection
12	reflection	4.0	reflection log / insights
13	introspection	4.0	runtime / cogni- tion self- inspection

#	name	timeout_s	accesses
14	knowledge_graph	3.0	entity / relation graph

Execution (AgentBus.dispatch, agent_bus.py:1533): - **Selective fan-out**: tiny filler chat → {memory, orchestrator}; non-chat → `_select_agents_for_intent` minimal set (keyword/action ladder, line 470); plain CHAT → broad fan-out across all `_enabled` agents. - **Parallel**: `ThreadPoolExecutor`, one thread per agent (or `runtime_policy.budget("agent_workers", floor=4, ceiling=32)`). - **Per-agent hard timeout**: `future.result(timeout=agent.timeout_s)` (1585); timeout/exception → failed `AgentResult`, never blocks the response. - **Aggregation** (`_aggregate_confidence`, 1248): per-agent contribution = `evidence_quality × evidence_density × learned per-(agent,action) calibration`; single-agent cap; empty-bus ceiling; corroboration bonus at ≥ 3 contributors. This part is genuinely solid.

When each runs (engine_gate_eli_orch_should_run, engine.py:8962)

Situation	Path
Quick-mode CHAT, or phatic	AgentBus directly (engine.py:8994)
Balanced/deep CHAT	AgentOrchestrator — own retrieval; bus <i>not</i> used
Non-CHAT action (any mode)	AgentOrchestrator → calls bus + ReAct loop
Orchestrator returns None / raises	Falls back to AgentBus directly

Planning artifacts — 4 of them, only 1 drives execution

1. **ReAct loop** (orchestrator.py:598) — the *only* planner that actually sequences execution (tool → observe → decide → next tool).
2. **PlannerAgent.plan_retrieval** (orchestrator.py:53) — plans *retrieval* budgets, not actions. Used every CHAT turn.
3. **Bus OrchestratorAgent plan** (agent_bus.py:1336) — emits an `orchestrator_plan` dict stored in `trace["orchestrator_plan"]` (engine.py:9026) for display / persona handoff / status. **Never executed.**
4. **execution_planner.build_execution_plan** (ExecutionPlan/PlanStep) — now **wired in** as the canonical typed plan. `AgentBus.dispatch` builds it each turn, injecting the proven `_select_agents_for_intent` result as `agent_profile`, and drives `active_agents` *through* `plan.agent_profile` (selection result unchanged). The plan is surfaced on `DispatchResult.execution_plan`. `EXECUTE_GOAL` also builds a real plan via it.
5. **task_planner.TaskPlanner** — no longer a hardcoded shim; it now delegates to `execution_planner.build_execution_plan` so there is a single real plan representation across the codebase.

Where it is actually weak (corrected)

1. **Two retrieval strategies — but NOT a real weakness (corrected).** Earlier framed as duplicate engines; on proper reading both bottom out on the *same* `eli/memory/memory.py` primitives (`recall_memory:1722`, `search_conversations:2533`). The bus's `BusMemoryAgent` is a thin wrapper over the all-in-one `recall_memory` (lightweight, for quick/parallel); the orchestrator's `OrchestratorMemoryAgent` decomposes the same primitives (`keyword_only=True`) + adds RAG + rerank (heavyweight, for deep). A core retrieval bug is fixed once in `memory.py` for both. The fast/deep split is deliberate; forcibly unifying would collapse it for no gain. Left as-is.
2. **The bus is still a single isolated round.** Within one dispatch, the 14 agents cannot consume each other's output. The ReAct loop chains *executor tool actions*, not bus agents — so a bus-level dependency (e.g. KG seeded by memory's entities) still cannot be expressed.

3. **Planning partially consolidated (improved).** `execution_planner` is now the canonical typed plan and drives bus selection; `task_planner` delegates to it. Remaining overlap: the engine's `_build_runtime_orchestrator_plan` (rich stage dict) and the bus `OrchestratorAgent` plan still coexist with the typed `ExecutionPlan`. The ReAct loop remains the only planner that actually *executes* a sequence. A future pass could fold the stage dict into `ExecutionPlan.steps` so there is one plan type end-to-end.
4. **ReAct loop fragility — FIXED.** Now validates the proposed `T00L:<action>` against `SUP-PORTED_ACTIONS` (143 registered actions) and stops the loop on an unknown/hallucinated action instead of switching to it; merges `intent["args"]` (preserving originals) instead of overwriting. (`orchestrator.py` ReAct block.)
5. **Custom-agent timeout override — FIXED.** The override is now `_apply_runtime_policy_timeouts()`, applied to built-ins and **re-applied after `_load_custom_agents()`**, so custom agents get hardware-adapted timeouts too.
6. **Timeouts don't cancel work.** `future.result(timeout)` only stops waiting; a timed-out write-capable agent (memory/habit) can still land a late DB write.
7. **Confidence coupled to a fixed evidence-key schema.** `_evidence_density` only counts known keys; a custom agent returning useful prose under an unknown key contributes 0.0 to grounding.
8. **No early-exit for direct actions; failures debug-only.** Bus waits on the slowest selected agent even when a direct `action_result` is already in hand; timeouts/exceptions log at `log.debug` with no health surface (despite the `agent_metrics` table existing).

Recommended fixes (prioritized)

Priority	Fix	Why
High	Unify retrieval: have the bus <code>BusMemoryAgent/KnowledgeGraphAgent</code> and the orchestrator's <code>OrchestratorMemoryAgent</code> call one shared retrieval module.	Kills the dual-stack divergence (#1) — the single biggest source of mode-dependent bugs.
High	Move <code>_load_custom_agents()</code> above the runtime-policy timeout loop (or re-apply the override after loading).	One-line fix for the redistribution timeout gap (#5).
High	Harden the ReAct loop: validate <code><action></code> against the known action set before chaining; merge (not overwrite) <code>intent["args"]</code> ; cap/parse defensively.	Closes silent-break + arg-loss (#4).
Med	Make the bus optionally two-round for dependent agents (round-1 results passed into round-2 <code>run()</code>), gated by a real plan.	Enables KG-seeded-by-memory etc. (#2).
Med	Either wire <code>execution_planner.ExecutionPlan</code> in as the bus's selection/sequencing source, or delete it + the shim <code>task_planner</code> to cut dead surface.	Resolves the 5-planner fragmentation (#3).

Priority	Fix	Why
Med	Generic evidence-density floor: count any non-empty payload at low weight so unknown-key/custom agents aren't zeroed.	Makes custom agents first-class to confidence (#7).
Med	Cooperative-cancel token checked before write-capable agents commit.	Closes the late-write hole (#6).
Low	Early-return once a direct action_result exists; surface agent health (timeout rate, p95) from agent_metrics in RUNTIME_STATUS.	Latency + observability (#8).

Highest leverage: **#1 (unify the two retrieval stacks)** and **#4 (harden the ReAct loop)** — those are where correctness actually drifts today. The bus's confidence math and timeout enforcement are already good and don't need work.

Update — 2026-06-09

- **HabitAgent ↔ proactive disconnect fixed.** The bus HabitAgent used to read only get_habit_rules() (the usually-empty habit_rules table), so the chat path was blind to the behaviour the proactive daemon detects into the habits table. It now also reads get_detected_habits() and emits a summary + detected_habits; AgentResult.has_evidence recognises the new keys (it was judging the agent "no evidence" → dropped), and DispatchResult.to_context_block renders the habit summary into the chat context. Habit data is now consistent across the chat agent, HABIT_STATUS, and the persona overlay.
- **Goal autogenesis** (planning/goal_autogenesis.py) feeds the autonomy/goal-tick stack from ELI's own signals — see runtime_planning_world.md.